The `took` attribute, measured in milliseconds, indicates the time it takes for the search request to complete. This time is measured from when a coordinator node receives the request to the time it aggregates the response before sending it back to the client. It doesn't include client-to-server marshaling/unmarshalling times.

The `timed_out` attribute is a Boolean flag that indicates whether the response has partial results, meaning whether any of the shards failed to respond in time. For example, if we have three shards and one fails to return results, the response consists of the results from the two shards but indicates the failed shard's state in the next object under the `_shards` attribute.

The `shards` attribute provides the number of shards that successfully executed the query and returned results and the number that failed. The `total` field is the number of shards expected to be searched, and the `successful` field denotes the shards that have returned data. A `failed` flag, on the other hand, indicates shards that failed during execution of the search query; the flag is denoted by the `failed` attribute.

The `hits` attribute (called *outer* `hits`) consists of information about the results. It contains another `hits` field (called *inner* `hits`). The outer `hits` object consists of the returned results, maximum score, and total results. The maximum score, represented by the `max_score` property, is the returned document with the highest score. The inner `hits` object consists of the results (the actual documents). This is an array of all the individual documents sorted by relevancy in descending order. Each document receives this `_score` attribute if the query is executed in a query context.

We've mentioned the two types of requests we can create for a query that asks a question: URI request and Query DSL. The next two sections discuss these in detail.

> **NOTE** As stated earlier, because of the versatility and features of Query DSL, we use it as our search mechanism in this book. However, for completeness, we briefly discuss URI request search in the following section. This way, you can experiment by creating equivalent URI request methods if you want to create cross-function search queries.

## 8.6 URI request searches

The URI request method is an easy way to search for simple queries. We invoke the `_search` endpoint by passing required parameters. The following syntax shows how to implement this search:

```
GET|POST <your_index_name>/_search?q=<name:value> AND|OR <name:value>
```

The `_search` endpoint is invoked on our index (or multiple indexes) with a query in the form of `q=<name:value>`. Note that the query is appended after the `_search` endpoint with a question mark (`?`) delimiter. With this method, we pass in the query parameters attached to the URL as `name:value` pairs. Let's issue some search queries using this method.